CampusSphere

Campus Opportunity Hub

UML Diagrams: Class, Object & Use Case

Rajwardhan Singh - 24CSB1A52
Ramitha Ramesh - 24CSB1A53
Rayi Charan Harsha - 24CSB1A54

Version 1.0

Date : 19-Oct-2026

1. Introduction

This document presents the three UML diagrams prepared for CampusSphere - Campus Opportunity Hub: the Class Diagram and Object Diagram (Structural / Logical View), and the Use Case Diagram (Use-Case View). Each section below leaves a marked, bordered space where the diagram image can be inserted, followed by a detailed paragraph explanation of that diagram.

2. Class Diagram

2.1 Diagram

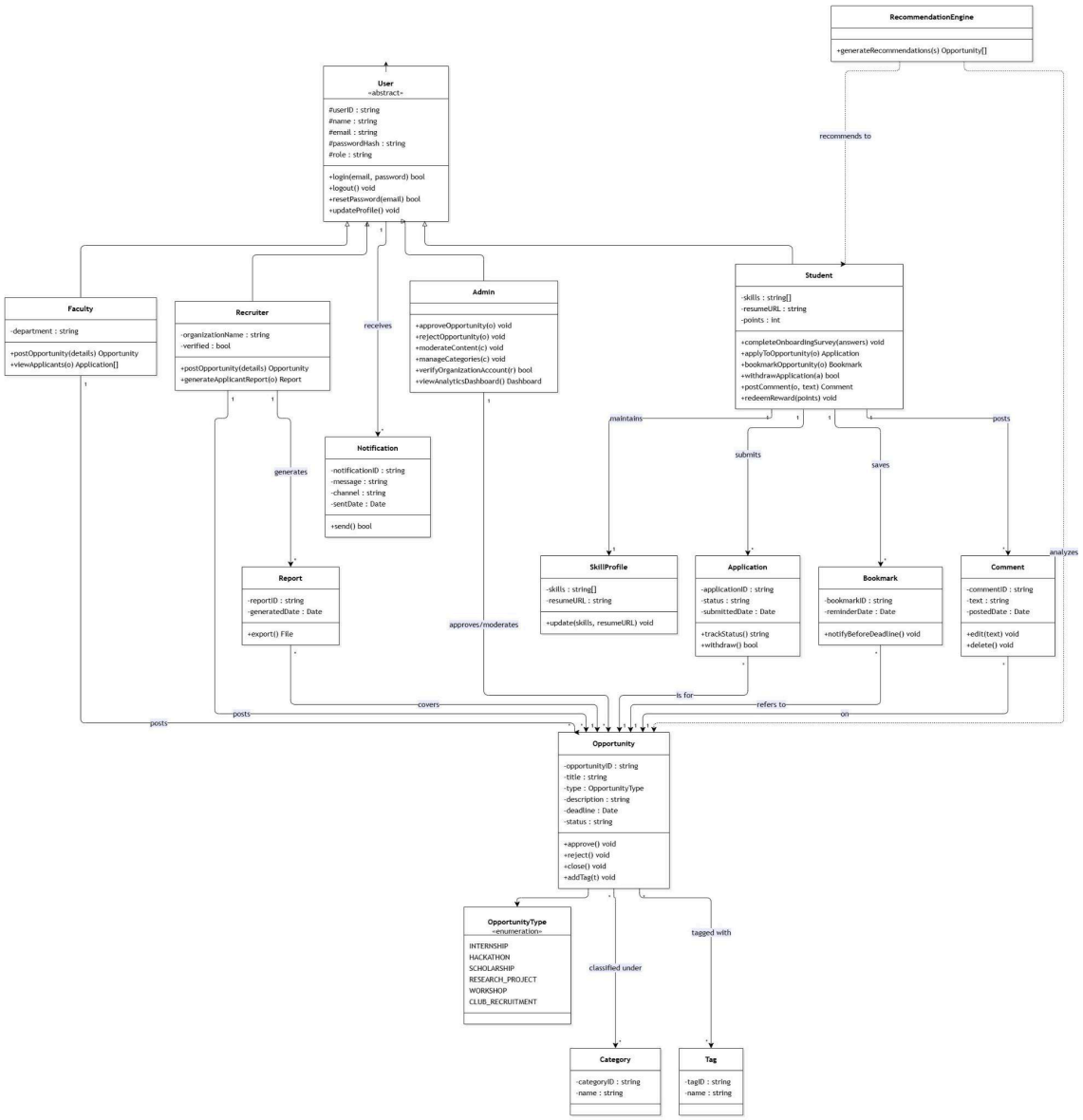


Figure 1: CampusSphere Class Diagram

2.2 Explanation

The Class Diagram captures the static, structural backbone of CampusSphere. At its root sits the abstract class User, which holds the attributes and behavior common to every person who logs into the platform – a unique ID, name, email, hashed password and role, along with the shared operations login(), logout(), resetPassword() and updateProfile(). Four concrete classes – Student, Faculty, Recruiter and Admin – specialize User through generalization (shown as hollow-triangle arrows pointing up to User), each adding only the attributes and methods specific to that role: Student carries skill/resume data and reward points and can complete the onboarding survey, apply, bookmark, comment and redeem rewards; Faculty and Recruiter both carry organization-specific attributes and can post opportunities, with Recruiter additionally able to generate applicant reports; Admin carries no extra attributes but owns the moderation and governance operations – approving, rejecting, verifying organizations and viewing analytics. The Opportunity class is the second pillar of the model, representing any internship, hackathon, scholarship, research project, workshop or club recruitment listing, distinguished by its type attribute which is constrained to the OpportunityType enumeration. Around these two pillars, a set of supporting classes model the rest of the workflow: Application links a Student to the Opportunity they applied for and tracks its status; SkillProfile holds the data a student maintains about themselves; Bookmark and Notification support saving-for-later and alerting; Category and Tag provide many-to-many classification of opportunities; Comment lets students discuss a posting; and Report is produced by an organizer for a specific posting's applicants. Finally, RecommendationEngine is connected to Student and Opportunity by dependency arrows (dashed, open arrowheads) rather than associations, because it reads and analyzes their data to produce recommendations without owning or being owned by either – a deliberate modeling choice that keeps the recommendation logic decoupled from the core domain classes. Multiplicities on every association (1, *, 1..1, etc.) pin down exactly how many instances of one class can be linked to instances of another – for example, one Student can have many Application and Bookmark objects, but each Application points to exactly one Opportunity.

3. Object Diagram

3.1 Diagram

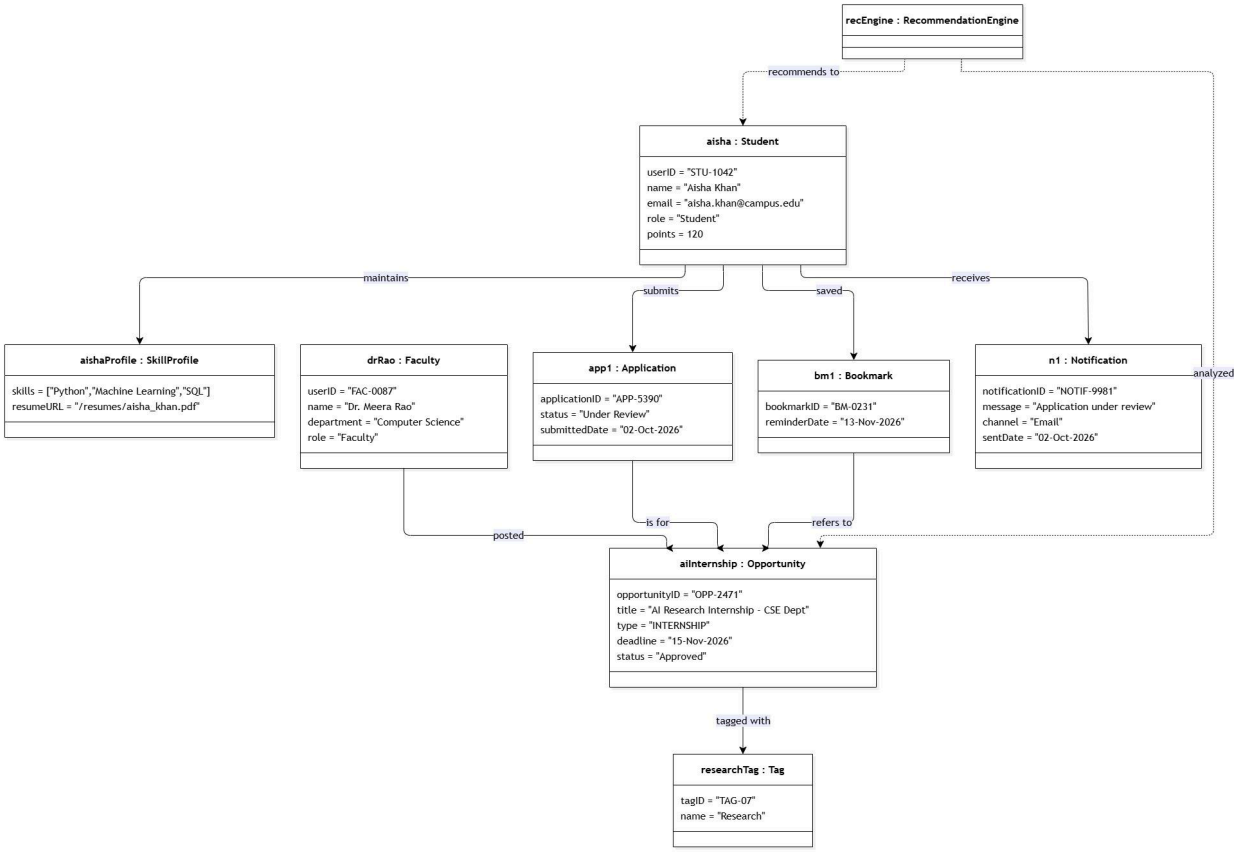


Figure 2: CampusSphere Object Diagram (run-time snapshot)

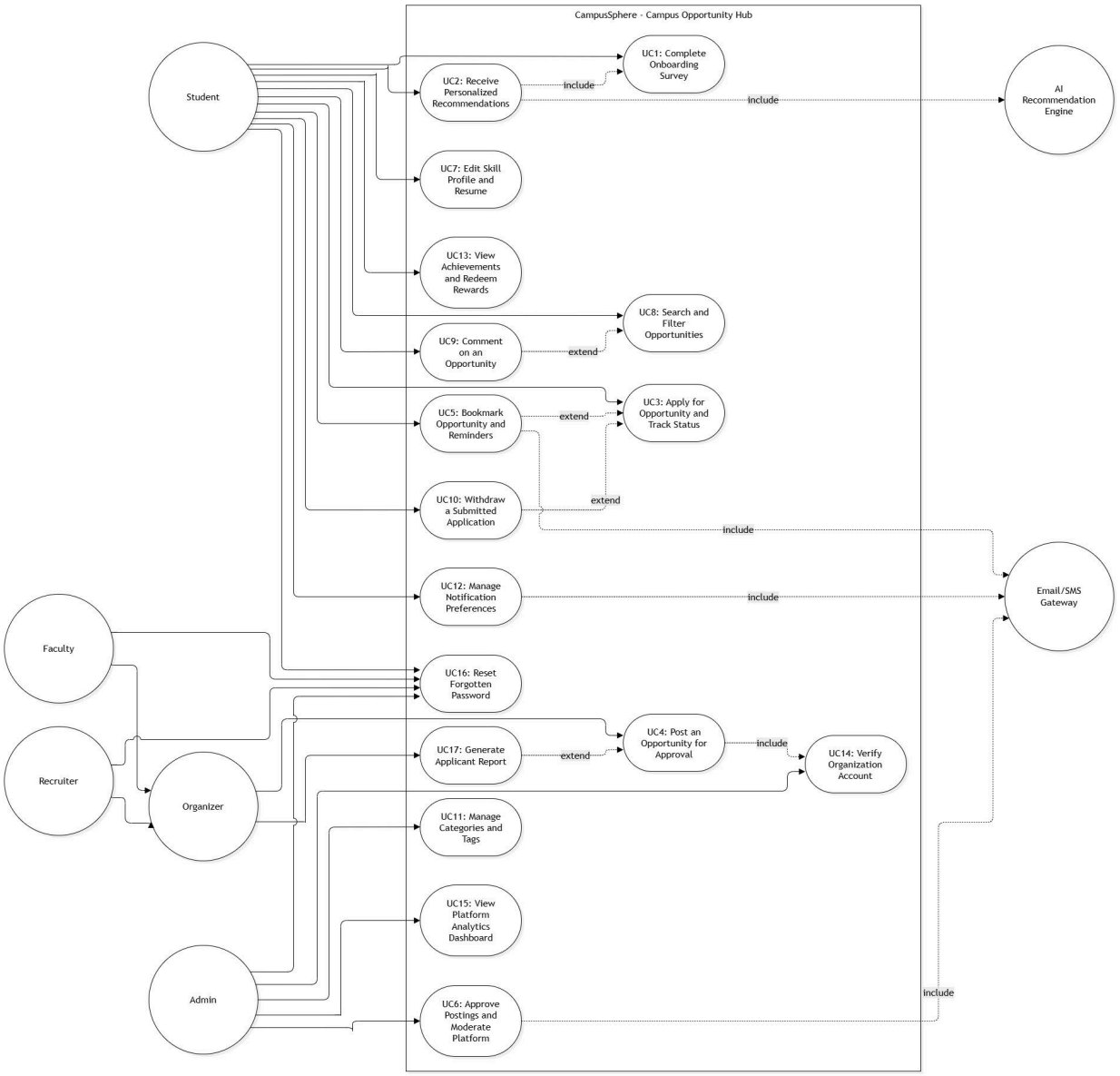
3.2 Explanation

Where the class diagram describes the system's structure in the abstract, the object diagram freezes that structure at one instant in time, using real instances with real data to prove the model actually works for a genuine scenario. The scenario chosen follows a student, Aisha Khan (aisha : Student, ID STU-1042), who has already built out her skill profile (aishaProfile : SkillProfile, listing Python, Machine Learning and SQL plus a resume link) and submitted an application (app1 : Application, status “Under Review”) for an opportunity titled “AI Research Internship – CSE Dept” (aiInternship : Opportunity, status “Approved”), which was posted by a faculty member, Dr. Meera Rao (drRao : Faculty). Aisha has also bookmarked the same opportunity with a reminder set for just before its deadline (bm1 : Bookmark), and has received an email notification about her application's status (n1 : Notification). The opportunity itself carries a classification tag (researchTag : Tag, “Research”), and in the background a recEngine : RecommendationEngine object is shown analyzing both Aisha's profile and this opportunity to generate her future recommendations. Every link drawn between these objects is a concrete instance of an association already defined in the class diagram – aisha → app1 instantiates the Student–Application association, app1 → aiInternship instantiates

Application–Opportunity, and so on – so the object diagram is best read side-by-side with the class diagram as a worked example confirming that a single student can simultaneously hold a profile, multiple applications, bookmarks and notifications exactly as the structural model allows. The naming convention for every object follows the UML standard instanceName : ClassName (e.g. aisha : Student), which visually distinguishes an object from a class even though the box shape is identical.

4. Use Case Diagram

4.1 Diagram



CampusSphere Use Case Diagram

Figure 3: CampusSphere Use Case Diagram

4.2 Explanation

The Use Case Diagram shifts perspective from internal structure to external behavior, showing CampusSphere purely from the point of view of the people and systems that use it. Six actors sit outside the system boundary: Student, Faculty, Recruiter and Admin are the four human roles, while the Email/SMS

Notification Gateway and AI Recommendation Engine are secondary, non-human actors representing external systems the platform depends on. A generalized actor, Organizer, sits between Faculty/Recruiter and the use cases they share – both roles inherit from it via generalization arrows, meaning anywhere Organizer connects to a use case (posting an opportunity, generating an applicant report), both Faculty and Recruiter can perform it, without having to draw duplicate lines from each. Inside the system boundary are the 17 use cases carried over from the CampusSphere Use Cases Document: students can complete the onboarding survey, receive recommendations, browse/search/apply/withdraw/bookmark/comment on opportunities, maintain their profile, manage notification preferences and redeem rewards; organizers post opportunities and pull applicant reports; and admins approve postings, moderate content, manage categories, verify organizations and view the analytics dashboard, with password reset available to every role. The dashed arrows with «include» labels mark mandatory sub-behavior that is always executed as part of the base use case – for instance, posting an opportunity always includes verifying the organizer's account first, and receiving recommendations always includes the data gathered during onboarding. The dashed arrows with «extend» labels mark optional behavior that only sometimes runs on top of a base use case – bookmarking, commenting, withdrawing an application and generating a report are all modeled this way, because a student or organizer may or may not trigger them while performing the base action. Finally, several use cases connect outward via «include» to the two external system actors, making explicit that functionality like recommendations, reminders, approval notifications and preference updates can't complete without those systems participating. Taken as a whole, the diagram answers “who can do what” for CampusSphere at a glance, while the include/extend relationships capture the dependencies and optionality that a plain actor-to-use-case line can't express on its own.